Mini-projet Weaviate : Architecture du Système et Modèle de Données

Mohamed Bedine

1 Contexte et objectif

Ce mini-projet met en place une base de données vectorielle **Weaviate** déployée avec **Docker Compose** afin d'implémenter une **recherche sémantique** sur une mini-base de connaissances de type FAQ. L'objectif est de démontrer un flux complet : (i) déploiement local de l'infrastructure, (ii) ingestion des données au format `JSON`, (iii) vectorisation automatique des textes via un service **Transformers** local, (iv) indexation dans Weaviate, et (v) exécution de requêtes *near_text* pour récupérer les réponses les plus pertinentes même lorsque la formulation de la question change.

2 Architecture générale du système

L'architecture repose sur deux services Docker principaux : **Weaviate** (la base de données) et **Transformers Inference** (le service qui calcule les embeddings). Le script Python joue le rôle d'orchestrateur applicatif : il crée la collection, charge les données depuis `data.json`, insère les objets en batch, puis exécute des requêtes sémantiques. La persistance est assurée par un volume Docker (`weaviate_data`) permettant de conserver les objets, leurs vecteurs et l'index entre deux redémarrages.

Avant l'insertion, le schéma doit exister : on définit une collection (ici `FAQ`) avec ses propriétés textuelles. Au moment de l'ingestion, Weaviate envoie le texte au service Transformers afin d'obtenir un vecteur (embedding). Une fois reçu, Weaviate stocke l'objet, associe le vecteur, puis construit ou met à jour l'index vectoriel. Lors d'une recherche, la requête utilisateur est elle aussi vectorisée, et Weaviate renvoie les objets dont les vecteurs sont les plus proches selon une mesure de similarité.

FAQ	
TEXT	question
TEXT	answer
TEXT	tag
VECTOR	embedding

FIGURE 1 – Modèle de données logique : collection `FAQ` avec propriétés textuelles et vecteur d'embedding.

3 Architecture interne côté base de données

La Figure 1 décrit la structure centrale stockée dans Weaviate. Chaque entrée de la collection `FAQ` contient des propriétés `question`, `answer` et `tag`. Les trois champs textuels représentent le contenu métier et les métadonnées. En parallèle, Weaviate conserve un champ **embedding** (vecteur) qui résume le sens du texte sous forme numérique. Ce vecteur est indispensable pour la recherche sémantique, car il permet d'effectuer des comparaisons mathématiques de similarité entre la requête et les objets stockés.

Le champ `tag` n'est pas obligatoire pour la recherche sémantique, mais il est très utile dans un contexte réel. Il sert à classer les objets par thème (par exemple `docker`, `concept`, `search`) et permet ensuite de filtrer ou restreindre la recherche à un sous-ensemble cohérent. Ainsi, si la base grandit (plusieurs domaines, plusieurs cours, plusieurs documents), on peut effectuer une recherche sémantique *dans un thème* sans mélanger les sujets.

4 Pipeline d'ingestion et de recherche

La Figure 2 illustre le déroulement complet depuis le script Python jusqu'à l'obtention des résultats. Le script commence par créer la collection `FAQ` (schéma). Ensuite, il charge les objets depuis `data.json` et lance une insertion en batch. Pour chaque objet, Weaviate appelle le service Transformers pour calculer l'embedding. Une fois le vecteur retourné, Weaviate enregistre l'objet, stocke le vecteur et met à jour l'index, ce qui rend l'information immédiatement interrogeable.

Pour la recherche, le script envoie une requête *near_text*. Weaviate vectorise la requête via le même service Transformers, puis recherche les voisins les plus proches dans l'espace vectoriel. Le résultat final est un *Top-1* ou *Top-K* d'objets retournés au script (question, réponse, tag). Ce mécanisme permet de retrouver la bonne réponse même si les mots exacts diffèrent, tant que le sens global reste proche.

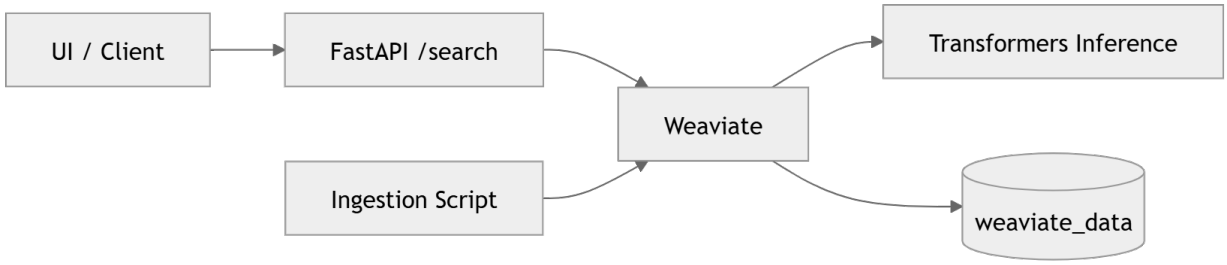

FIGURE 2 – Séquence d'exécution : création du schéma, ingestion batch, vectorisation, indexation et recherche *near_text*.

5 Architecture cible conforme pour une mini-application

Une évolution naturelle de ce mini-projet consiste à ajouter une couche **API** (par exemple **FastAPI**) afin d'exposer un endpoint `/search`. Dans ce cas, l'utilisateur (UI ou client) envoie sa requête à l'API, l'API interroge Weaviate, et Weaviate s'appuie sur Transformers pour la vectorisation. En parallèle, l'ingestion peut être séparée dans un script dédié, ce qui correspond à une architecture plus proche d'un système de production : un composant *Ingestion* alimente la base, tandis qu'un composant *Serving* (API) répond aux requêtes.

Cette architecture rend l'application facilement intégrable dans un dashboard, un chatbot, ou un moteur de recherche interne. Elle est aussi plus maintenable : on peut planifier des ré-ingestions, versionner les données, ajouter des filtres métiers, ou étendre à des documents plus complexes (PDF découpés en chunks, rapports techniques, tickets, etc.).

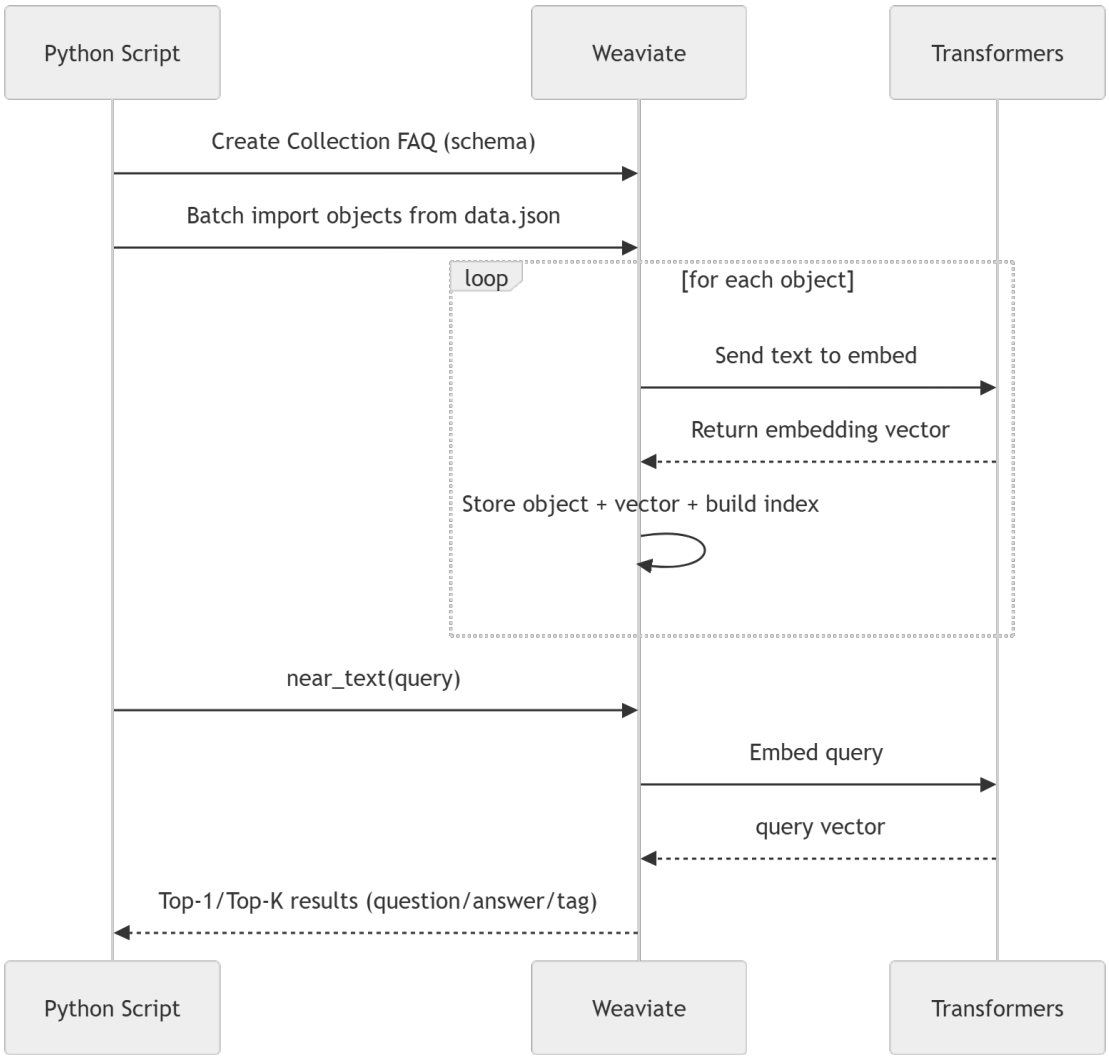

FIGURE 3 – Architecture cible : séparation ingestion / API, Weaviate au centre, Transformers pour l'embedding, et volume de persistance.

6 Conclusion

Ce mini-projet démontre une chaîne complète de **recherche sémantique** : déploiement Docker, ingestion JSON, vectorisation locale via Transformers, stockage et indexation dans Weaviate, puis interrogation *near_text*. La présence d'un champ `tag` illustre l'ajout de métadonnées pour structurer la base et préparer des filtres thématiques. Enfin, l'architecture cible proposée montre comment passer d'un script de test à une mini-application exploitable via une API, ouvrant la voie à des cas d'usage RAG, FAQ intelligente et recherche documentaire.